

PROCESSO SELETIVO Bry

RELATÓRIO DO DESAFIO TÉCNICO

Desenvolvedor Back-End Java

CANDIDATO

Henrique Mateus Teodoro

Sumário

1	Introdução	1
2	Ambiente e tecnologias	1
3	Estrutura e arquitetura do projeto	1
3.1	API e linha de comando no mesmo executável	2
3.2	Provider criptográfico	2
3.3	Tratamento de erros no domínio	2
3.4	Relógio injetável	2
4	Implementação das etapas	3
4.1	Etapa 1 — Resumo criptográfico (SHA-512)	3
4.2	Etapa 2 — Assinatura digital	3
4.2.1	Leitura do PKCS12	3
4.2.2	Geração da assinatura	3
4.3	Etapa 3 — Verificação da assinatura	4
4.3.1	Integridade	4
4.3.2	Confiança do certificado	5
4.3.3	Resultado e informações do signatário	5
4.4	Etapa 4 — API REST	6
4.4.1	Endpoint de assinatura	6
4.4.2	Endpoint de verificação	6
4.4.3	Tratamento de erros	7
4.5	Etapa 5 — Relatório das atividades	8
4.6	Etapa 6 — Distribuição do código	8
5	Compilação, execução e artefatos	8
5.1	Com Docker	8
5.2	Com Java 17	8
5.3	Artefatos	9
6	Testes e qualidade	9
7	Integração contínua e distribuição	10
7.1	Integração contínua	10
7.2	Distribuição	10
8	Dificuldades encontradas	10
9	Limitações e melhorias futuras	11

10 Repositório do projeto	11
11 Conclusão	12

1 Introdução

Este relatório apresenta o desenvolvimento do desafio técnico para a vaga de Desenvolvedor Back-End Java da Bry. O desafio é composto por quatro etapas práticas: o cálculo do resumo criptográfico de um documento, a geração de uma assinatura digital, a verificação dessa assinatura e a disponibilização das operações em uma API REST. A quinta etapa é este relatório, e a sexta, opcional, é a distribuição do código por um pipeline.

Todas as etapas foram implementadas, incluindo a opcional. O projeto gera o hash SHA-512 do documento, produz uma assinatura CMS attached com SHA-512 e RSA, verifica a integridade da assinatura e a confiança do certificado na cadeia fornecida e expõe as operações de assinatura e verificação na API. A cada versão, um pipeline publica o executável, os arquivos das etapas 1 e 2 e uma imagem Docker.

Além de cumprir as etapas, o objetivo foi entregar um código robusto, organizado e fácil de executar, com testes automatizados, tratamento de erros consistente e um ambiente padronizado com Docker.

2 Ambiente e tecnologias

O desenvolvimento foi realizado em Linux (Ubuntu 24.04). As tecnologias utilizadas foram:

- **Java 17** (17.0.20), conforme exigido pelo enunciado;
- **Spring Boot 3.5.16**, a versão mais recente da linha 3.x;
- **BouncyCastle 1.86** (bcprov-jdk18on e bcpkix-jdk18on);
- **Maven**, com o Maven Wrapper (./mvnw) fixando a versão 3.9.16;
- **JUnit 5**, **AssertJ**, **MockMvc** e **JaCoCo** para testes e cobertura;
- **Docker** e **Docker Compose** para padronizar a compilação e a execução;
- **GitHub Actions** para integração contínua e publicação das versões.

O pom.xml também usa o plugin **maven-enforcer** para impedir a compilação com outra versão do Java, evitando diferenças entre ambientes.

3 Estrutura e arquitetura do projeto

O código está organizado em quatro pacotes, separando o núcleo criptográfico das formas de uso (API e linha de comando):

```
src/main/java/br/com/bry/challenge/
|-- api/           Endpoints REST, resposta da verificação e tratamento de
                  erros
|-- cli/           Execução das etapas 1, 2 e 3 pela linha de comando
|-- config/        Configurações da aplicação
'-- crypto/        Hash, PKCS12, assinatura e verificação CMS
```

As principais classes do pacote **crypto** são:

- **DigestService**: cálculo de hash;

- `Pkcs12CredentialLoader` e `SigningCredential`: leitura da chave privada e do certificado;
- `CmsSigner`: geração da assinatura CMS;
- `CmsVerifier` e `VerificationResult`: verificação da assinatura e seu resultado;
- `CertificateTrustValidator`: validação da cadeia de confiança;
- `CryptoException` e `CryptoErrorCode`: erros do domínio.

O mesmo núcleo é usado pela API e pela linha de comando, sem duplicação de código. A seguir estão as principais decisões de organização.

3.1 API e linha de comando no mesmo executável

As etapas 1, 2 e 3 são executadas por um perfil do Spring, o `cli`. Com esse perfil ativo, a aplicação não sobe o servidor web. A classe `ChallengeStepsRunner` executa as três etapas, grava os resultados e a aplicação encerra. Sem o perfil, a aplicação sobe a API. Assim, um único jar e uma única imagem Docker atendem aos dois usos, e a linha de comando reaproveita a mesma configuração e injeção de dependências da API.

3.2 Provider criptográfico

Uma única instância do `BouncyCastleProvider` é criada como bean do Spring e injetada nas classes que fazem as operações criptográficas, que a usam de forma explícita. Com isso, não foi necessário registrar o provider globalmente com `Security.addProvider`, o que alteraria o estado de toda a JVM.

3.3 Tratamento de erros no domínio

Os erros das operações criptográficas são representados por uma única exceção, `CryptoException`, acompanhada de um código do enum `CryptoErrorCode` (por exemplo, `INVALID_PASSWORD` ou `INVALID_SIGNATURE_FORMAT`). Essa abordagem permite tratar cada tipo de erro de forma específica sem criar uma classe de exceção para cada caso. Na API, cada código é convertido para o status HTTP adequado.

Uma assinatura bem formada, mas inválida, não é tratada como erro. Nesse caso a verificação retorna um resultado negativo com os motivos. Exceções ficam reservadas para entradas que não podem ser processadas, como um arquivo que não é uma assinatura CMS.

3.4 Relógio injetável

O momento da assinatura e a data da verificação são obtidos de um `java.time.Clock` injetado, portanto ele usa o relógio do sistema. Nos testes, um relógio fixo torna os resultados determinísticos. Isso evita, por exemplo, que os testes passem a falhar quando o certificado do desafio expirar.

4 Implementação das etapas

4.1 Etapa 1 — Resumo criptográfico (SHA-512)

A classe `DigestService` calcula o hash a partir de um `InputStream`, lendo o conteúdo em blocos, sem carregar o arquivo inteiro em memória. As versões que recebem um arquivo (`Path`) ou bytes em memória apenas adaptam a entrada e chamam esse mesmo método, de modo que existe um único caminho de cálculo. O algoritmo é um parâmetro, o que permite reaproveitar o serviço na verificação da assinatura com o algoritmo informado pela própria assinatura.

O resultado é gravado em `artifacts/doc.txt.sha512`, no mesmo formato do comando `sha512sum`, o que permite conferi-lo de forma independente com `sha512sum -c`. O hash obtido para o documento é apresentado abaixo, dividido em duas linhas:

```
dc1a7de77c59a29f366a4b154b03ad7d99013e36e08beb50d976358bea7b0458
84fe72111b27cf7d6302916b2691ac7696c1637e1ab44584d8d6613825149e35
```

4.2 Etapa 2 — Assinatura digital

4.2.1 Leitura do PKCS12

A classe `Pkcs12CredentialLoader` lê o arquivo PKCS12 com a classe `KeyStore` do Java, como pede o enunciado, e retorna um `SigningCredential` com a chave privada e a cadeia de certificados. Ela oferece dois métodos com regras explícitas:

- `load(pkcs12, senha, alias)`: usa exatamente a chave do alias informado (usado pela linha de comando);
- `load(pkcs12, senha)`: exige que o arquivo tenha exatamente uma chave privada (usado pela API, que recebe um PKCS12 qualquer).

Se o alias não existir, ou se houver várias chaves e nenhum alias, a operação falha com uma mensagem que lista as chaves disponíveis. Também são validados a senha, o formato do arquivo e o tipo da chave, que precisa ser RSA. Uma senha incorreta é diferenciada de um arquivo corrompido, para que a mensagem de erro seja precisa.

Importante: Durante a implementação, foi identificado que o alias informado no enunciado `"e2618a8b-20de-4dd2-b209-70912e3177f4"` não corresponde ao alias gravado no arquivo, que está entre chaves: `"{e2618a8b-20de-4dd2-b209-70912e3177f4}"`. A diferença foi confirmada com a ferramenta `keytool` do Java. A configuração da aplicação usa o alias real, presente no arquivo.

4.2.2 Geração da assinatura

A classe `CmsSigner` gera a assinatura com as classes do BouncyCastle:

1. `JcaContentSignerBuilder` com o algoritmo `SHA512withRSA`;

2. `JcaSignerInfoGeneratorBuilder`, que monta as informações do signatário e os atributos assinados;
3. `JcaCertStore`, que embute o certificado do signatário na assinatura;
4. `CMSSignedDataGenerator`, com o parâmetro de encapsulamento ativado, o que torna a assinatura attached;
5. codificação final em DER.

O atributo `signingTime` é criado explicitamente a partir do relógio da aplicação, já que a data da assinatura é uma das informações exigidas na verificação. Os atributos `contentType`, `messageDigest` e `cmsAlgorithmProtection` são adicionados pelo BouncyCastle.

O resultado é gravado em `artifacts/doc.txt.p7s`. A assinatura foi conferida de forma independente com o OpenSSL (`openssl cms -verify`), que confirmou a assinatura e a cadeia de confiança. A estrutura ASN.1 do arquivo mostra os elementos esperados:

```
OBJECT      :pkcs7-signedData
OBJECT      :sha512                      (algoritmo de hash)
OBJECT      :pkcs7-data
OCTET STRING :Teste vaga back-end Java    (documento attached)
OBJECT      :contentType
OBJECT      :signingTime
UTCTIME      :261007034340Z
OBJECT      :1.2.840.113549.1.9.52      (cmsAlgorithmProtection)
OBJECT      :messageDigest
OBJECT      :sha512WithRSAEncryption    (algoritmo de assinatura)
```

O BouncyCastle 1.86 registra o algoritmo de assinatura como `sha512WithRSAEncryption`, e não como `rsaEncryption`. As duas formas são aceitas pela RFC 5754.

4.3 Etapa 3 — Verificação da assinatura

4.3.1 Integridade

A classe `CmsVerifier` recebe a assinatura em Base64 ou em DER (o Base64 tem prioridade). O conteúdo é decodificado como Base64 e, se não for um Base64 válido, é tratado como DER.

A verificação ocorre em duas fases. Na primeira, toda a estrutura CMS é decodificada de uma vez: o signatário, o conteúdo, os certificados, o algoritmo e o `signingTime`. O BouncyCastle lê algumas partes da estrutura apenas quando elas são acessadas e sinaliza dados malformados com exceções não cheçadas. Por isso, qualquer falha dessa fase é convertida no erro `INVALID_SIGNATURE_FORMAT`. Também são rejeitadas assinaturas detached (sem o documento) e assinaturas com mais de um signatário. Na segunda fase, a verificação trabalha apenas com dados já decodificados.

A integridade é verificada com `SignerInformation.verify`, que confere a assinatura RSA sobre os atributos assinados, compara o atributo `messageDigest` com o hash do conteúdo e verifica se o certificado era válido na data declarada em `signingTime`. Cada falha gera um motivo específico (documento alterado, assinatura

que não corresponde à chave do certificado ou certificado fora da validade na data da assinatura).

O enunciado relaciona o hash do documento ao `encapContentInfo`. Esse campo contém o próprio documento attached, então a implementação extrai o conteúdo e calcula seu hash com o `DigestService`, usando o algoritmo indicado no `digestAlgorithm` da assinatura.

4.3.2 Confiança do certificado

A classe `CertificateTrustValidator` carrega os certificados de `resources/cadeia/` e valida o certificado do signatário com o algoritmo PKIX do Java (`CertPathBuilder`), usando o provider do BouncyCastle. Os certificados autoassinados da pasta são usados como âncoras de confiança (AC raiz), e os demais como ACs intermediárias. Os certificados embutidos na assinatura também podem completar o caminho, mas a confiança continua dependendo das raízes configuradas.

A cadeia é carregada na inicialização da aplicação. Se os certificados não puderem ser lidos, a aplicação não sobe, já que sem a cadeia nenhuma assinatura poderia ser validada.

Duas decisões foram tomadas nesta etapa:

- **Data de referência:** a confiança é avaliada na data atual. O `signingTime` é declarado pelo próprio signatário e não é autenticado por terceiros, então não serve como referência confiável sem um carimbo do tempo. Essa abordagem segue a validação básica descrita na norma ETSI EN 319 102-1 e a política de assinatura de referência básica (AD-RB) da ICP-Brasil. Assim, uma assinatura feita com um certificado que já expirou deixa de ser válida, mesmo que tenha sido feita dentro da validade.
- **Revogação:** a consulta às listas de certificados revogados (LCR) está desabilitada, pois exigiria acesso à rede para obter as listas publicadas pela AC.

4.3.3 Resultado e informações do signatário

O resultado da verificação é representado pelo record `VerificationResult`. Uma assinatura é considerada válida quando está íntegra e o certificado é confiável. O enunciado define a verificação da integridade e a da confiança como partes da etapa 3; por isso, o resultado final considera as duas, e cada uma também é apresentada separadamente.

Na linha de comando, a etapa 3 verifica o arquivo gerado na etapa 2 e imprime as informações relevantes:

```
Etapa 3 - Verificação da assinatura (artifacts/doc.txt.p7s)
Integridade:           true
Certificado confiável: true
Resultado:             VÁLIDA
Data da assinatura:    2026-10-07T03:43:40Z
Algoritmo de hash:    SHA-512
Hash do documento:    dc1a7de77c59a29f...825149e35
```

```
Signatário (CN):      HUB2 TESTES
Titular:              CN=HUB2 TESTES,OU=Validado por email,...
Emissor:              ...,CN=AC BRy Servidor Seguro v3
Número de série:      25F
Validade:             2021-07-21T00:00:00Z a 2029-07-21T18:22:00Z
Caminho de certificação: 1. CN=HUB2 TESTES,...
                       2. ...,CN=AC BRy Servidor Seguro v3
                       3. ...,CN=Autoridade Certificadora Raiz BRy ... v3
```

Se a assinatura gerada não for válida, a execução termina com erro.

4.4 Etapa 4 — API REST

A API é implementada pela classe `SignatureController`, que apenas converte as requisições HTTP para o domínio e reaproveita as classes das etapas anteriores. Os endpoints são exatamente os do enunciado, `POST /signature/` e `POST /verify/`, e ambos recebem `multipart/form-data`.

4.4.1 Endpoint de assinatura

O endpoint `/signature/` recebe três campos:

- `file`: arquivo a ser assinado;
- `pkcs12`: arquivo PKCS12 com a chave privada e o certificado;
- `password`: senha do PKCS12.

A resposta contém apenas a assinatura CMS em Base64, com o tipo `text/plain`, como descrito no enunciado. A senha é recebida como um campo do formulário multipart, e nunca como parâmetro da URL, convertida para `char[]` e apagada da memória após o uso.

4.4.2 Endpoint de verificação

O endpoint `/verify/` recebe o arquivo assinado no campo `signature`. O enunciado pede o arquivo em Base64; como complemento, também é aceito o arquivo `.p7s` binário (DER). Em contato anterior com a equipe da Bry sobre o mesmo endpoint, foi orientado que o suporte a Base64 é obrigatório e que aceitar os dois formatos seria um complemento desejável.

A resposta contém os campos obrigatórios `status` e `infos`, além de informações adicionais em `details`:

```
{
  "status": "VALIDO",
  "infos": {
    "signerName": "HUB2 TESTES",
    "signingTime": "2026-10-07T03:43:40Z",
    "documentHash": "dc1a7de77c59a29f...825149e35",
    "digestAlgorithm": "SHA-512"
  },
  "details": {
```

```

    "integrityValid": true,
    "certificateTrusted": true,
    "certificate": {
      "subject": "CN=HUB2 TESTES,OU=Validado por email,...",
      "issuer": "...,CN=AC BRy Servidor Seguro v3",
      "serialNumber": "25F",
      "notBefore": "2021-07-21T00:00:00Z",
      "notAfter": "2029-07-21T18:22:00Z"
    },
    "certificationPath": ["CN=HUB2 TESTES,...", "...", "..."],
    "failureReasons": []
  }
}

```

Quando a assinatura é bem formada, mas inválida (por exemplo, com o documento alterado ou com um certificado fora da cadeia confiável), a requisição foi processada corretamente e a API retorna HTTP 200 com `status` igual a `INVALIDO` e os motivos em `failureReasons`.

4.4.3 Tratamento de erros

A classe `ApiExceptionHandler` centraliza o tratamento de erros. Todas as respostas de erro seguem o formato Problem Details (RFC 9457), com mensagens em português e o campo `code`, que identifica o motivo de forma estável:

```

{
  "type": "about:blank",
  "title": "Bad Request",
  "status": 400,
  "detail": "Senha do PKCS12 incorreta",
  "instance": "/signature/",
  "code": "INVALID_PASSWORD"
}

```

HTTP	Situação (code)
400	campo ausente (<code>MISSING_FIELD</code>), arquivo vazio (<code>EMPTY_FILE</code>), PKCS12 inválido, senha incorreta, chave ausente ou não RSA, conteúdo que não é uma assinatura CMS e algoritmo não suportado
413	arquivo acima do limite de 10 MB (<code>FILE_TOO_LARGE</code>)
415	requisição que não é <code>multipart/form-data</code> (<code>UNSUPPORTED_MEDIA_TYPE</code>)
500	falha ao gerar a assinatura (<code>SIGNATURE_FAILED</code>) e erros inesperados (<code>INTERNAL_ERROR</code>)

Em erros inesperados, os detalhes técnicos são registrados apenas no log, sem serem expostos ao cliente. Os logs da API registram as operações e seus resultados, sem incluir a senha nem os nomes dos arquivos enviados.

Os endpoints foram testados por testes de integração, com `curl` e com o Postman, recomendado no enunciado. A coleção do Postman está disponível em `docs/bry-challenge.postman_collection.json`.

4.5 Etapa 5 — Relatório das atividades

A quinta etapa é representada por este documento, que descreve a implementação, a estrutura do projeto, as instruções de execução, as decisões tomadas e as dificuldades encontradas. As instruções de uso também estão no README do repositório.

4.6 Etapa 6 — Distribuição do código

A etapa opcional foi implementada com o GitHub Actions. Ao enviar uma tag de versão (por exemplo, `v1.0.0`), o pipeline executa toda a integração contínua e, se tudo passar, publica uma release com o jar e os arquivos das etapas 1 e 2, além de uma imagem Docker no GitHub Container Registry. O funcionamento é detalhado na seção 7.2.

5 Compilação, execução e artefatos

Os comandos completos estão no README. Há duas formas de executar o projeto: com Docker, sem precisar de Java instalado, ou com Java 17 e o Maven Wrapper. Os comandos devem ser executados a partir da raiz do projeto.

5.1 Com Docker

```
docker compose up --build      # sobe a API na porta 8080
docker compose run --rm cli    # executa as etapas 1, 2 e 3
```

O `Dockerfile` tem duas fases. Na primeira, o projeto é compilado e testado dentro de um container com o JDK 17. Na segunda, a imagem final recebe apenas o JRE, o jar e os arquivos do desafio, e a aplicação é executada com um usuário sem privilégios de administrador. O serviço `cli` do Docker Compose grava os arquivos na pasta `artifacts/` do projeto com o usuário da máquina.

5.2 Com Java 17

```
./mvnw clean verify           # compila e testa
./mvnw spring-boot:run        # sobe a API
./mvnw spring-boot:run -Dspring-boot.run.profiles=cli # etapas 1, 2 e 3
```

5.3 Artefatos

- `artifacts/doc.txt.sha512`: hash SHA-512 do documento (etapa 1);
- `artifacts/doc.txt.p7s`: assinatura CMS attached em DER (etapa 2).

Os caminhos dos arquivos, o alias e a senha do PKCS12 ficam no `application.yml` e podem ser alterados por argumentos ou variáveis de ambiente.

6 Testes e qualidade

O projeto possui 92 testes automatizados: 73 de unidade e 19 de integração. Os testes de integração sobem a aplicação com o Spring e são identificados pelo sufixo `IntegrationTest`.

Classe de teste	Testes	Principais cenários
<code>DigestServiceTest</code>	9	vetores do NIST, arquivo do desafio, entrada vazia, equivalência entre arquivo, bytes e stream, falhas de leitura
<code>Pkcs12CredentialLoaderTest</code>	12	alias, arquivo com várias chaves, senha incorreta, arquivo inválido ou vazio, ausência de chave, chave EC
<code>SigningCredentialTest</code>	3	cadeia imutável e cadeia vazia
<code>CmsSignerTest</code>	9	conteúdo attached, algoritmos, atributos assinados, certificado embutido, validade criptográfica, DER
<code>CmsVerifierTest</code>	16	Base64 e DER, documento e assinatura alterados, chave que não corresponde ao certificado, signatário não confiável, certificado expirado, formatos inválidos
<code>CertificateTrustValidatorTest</code>	8	cadeia da Bry, AC desconhecida, intermediária ausente ou embutida, período de validade, cadeia inválida
<code>VerificationResultTest</code>	2	nome do signatário
<code>ChallengeStepsRunnerTest</code>	6	arquivos e saída das etapas 1, 2 e 3 e falhas de configuração
<code>ApiExceptionHandlerTest</code>	8	status HTTP de cada código de erro e erro inesperado
<code>SignatureControllerIntegrationTest</code>	16	assinatura, verificação em Base64 e DER, assinaturas inválidas e erros da API
Demais testes de integração	3	inicialização da aplicação, perfil <code>cli</code> e limite de upload

Os casos de erro são construídos com certificados e arquivos PKCS12 gerados em memória durante os testes, sem adicionar arquivos binários ao repositório.

A cobertura é medida pelo JaCoCo, e o build falha se ela ficar abaixo de 90% das linhas ou de 80% dos branches. A cobertura atual é de 97,6% das linhas e 91,2% dos branches. As linhas não cobertas correspondem a tratamentos de falhas internas que não podem ser provocadas com dados reais.

7 Integração contínua e distribuição

7.1 Integração contínua

O workflow de CI é executado a cada push e possui dois jobs, executados em Linux:

1. **Build e testes:** compila o projeto, executa os testes e verifica a cobertura; executa as etapas 1, 2 e 3 e confere os resultados com ferramentas independentes do projeto: `sha512sum` para o hash e `openssl cms -verify` para a assinatura e a cadeia de confiança;
2. **Imagem Docker e smoke test:** constrói a imagem, sobe o container, aguarda a aplicação ficar saudável e chama os endpoints `/signature/` e `/verify/`, exigindo o status `VALIDO`.

O cache das dependências do Maven e das camadas da imagem reduz o tempo das execuções seguintes, e execuções antigas da mesma branch são canceladas quando um novo commit é enviado.

7.2 Distribuição

O workflow de release é disparado por tags no formato `vX.Y.Z`. Ele reaproveita o workflow de CI, sem duplicar os passos de build e teste, e só publica a versão se toda a CI passar. Em seguida:

- confere se a versão da tag corresponde à versão do `pom.xml`;
- publica uma release no GitHub com o jar testado pela CI e os arquivos das etapas 1 e 2;
- publica a imagem Docker no GitHub Container Registry, com a tag da versão e a tag `latest`.

Com a imagem, a API pode ser executada sem clonar o repositório:

```
docker run -p 8080:8080 ghcr.io/henrique1803/bry-java-backend-challenge:1.0.0
```

8 Dificuldades encontradas

A primeira dificuldade foi a divergência entre o alias do certificado informado no enunciado e o alias gravado no arquivo PKCS12. Ao usar o valor do enunciado, a chave não era encontrada. A causa foi identificada inspecionando o arquivo com o `keytool`, com o alias real estando entre chaves.

Outra dificuldade foi tornar a verificação robusta para arquivos malformados. A leitura inicial da assinatura já tratava conteúdos inválidos, mas, ao testar a verificação com assinaturas corrompidas em posições variadas, algumas falhas ainda geravam exceções não tratadas. A causa é que o BouncyCastle lê partes da estrutura, como a chave pública do certificado e os signatários, apenas quando elas são acessadas. Na API, essas falhas resultariam em erro 500. A solução foi separar a verificação em

uma fase de leitura completa da estrutura, que converte qualquer falha em erro de formato, e uma fase de verificação sobre os dados já decodificados.

Também foi necessário definir a data de referência da validação do certificado. A primeira implementação usava o `signingTime`, mas, como esse atributo é declarado pelo próprio signatário, a validação passou a usar a data atual, de acordo com as normas de validação de assinaturas sem carimbo do tempo.

Por fim, a interpretação de alguns pontos do enunciado exigiu decisões, como o critério para o status `VALIDO`, o formato aceito pelo endpoint de verificação e o formato da resposta da assinatura. Essas decisões foram tomadas buscando seguir o enunciado e estão descritas nas seções correspondentes. A pouca experiência prévia com a biblioteca BouncyCastle também foi uma dificuldade, principalmente nas classes de geração e verificação de assinaturas CMS. Além disso, o prazo para a realização do desafio foi curto, de três dias, e coincidiu com dias de semana. Por isso, foi necessário conciliar o desenvolvimento do desafio com as tarefas do meu dia a dia.

9 Limitações e melhorias futuras

- **Revogação:** a consulta às LCRs (ou OCSP) não foi implementada, pois exigiria acesso à rede durante a verificação e os testes.
- **Validade de longo prazo:** sem um carimbo do tempo, uma assinatura deixa de ser válida quando o certificado expira. Para preservar a validade, seria necessário adotar uma política com carimbo do tempo, como a AD-RT da ICP-Brasil.
- **Senha na API:** a senha do PKCS12 é recebida pelo Spring como `String`, que não pode ser apagada da memória. A conversão para `char[]` limita sua permanência apenas no código da aplicação.
- **Políticas de assinatura:** a verificação não restringe algoritmos considerados fracos (como SHA-1) nem verifica o uso de chave do certificado.
- **Formatos de assinatura:** assinaturas detached e com mais de um signatário não são suportadas.
- **Segurança da API:** a API não possui autenticação, o que seria necessário em um ambiente de produção.

O projeto não utiliza banco de dados, já que nenhuma etapa do desafio exige persistência.

10 Repositório do projeto

O código-fonte, os testes, os artefatos e as instruções de execução estão disponíveis no repositório:

<https://github.com/Henrique1803/bry-java-backend-challenge>

11 Conclusão

O desafio foi concluído com todas as etapas solicitadas, incluindo a etapa opcional de distribuição. O projeto calcula e grava o hash SHA-512 do documento, gera uma assinatura CMS attached com SHA-512 e RSA, verifica sua integridade e a confiança do certificado na cadeia fornecida e apresenta as informações do signatário.

A API REST reutiliza as mesmas classes da linha de comando e oferece os endpoints de assinatura e verificação no formato solicitado, com tratamento de erros padronizado. A implementação é validada por 92 testes automatizados, por um pipeline que também confere os resultados com ferramentas independentes e por uma imagem Docker publicada a cada versão.